

L02 (OPTION B)

Group 8: Sydney Chilson, Chad Jones, Bryton McClintock, Natalia Solorzano, Hasan Zaidi

ITAI-3377

1/28/26

Conceptual Report: TensorFlow Lite Model Conversion

Introduction

The goal of this lab was to understand how a machine learning model can be prepared for deployment using TensorFlow Lite. TensorFlow Lite is designed for running models on devices with limited resources, such as mobile phones and embedded systems. In this activity, we converted an existing TensorFlow model into TensorFlow Lite format and tested it to ensure it still worked correctly.

Model Preparation

We began by working with a trained TensorFlow model. Before conversion, we verified that the model loaded correctly and could perform inference. This step ensured the model was compatible with TensorFlow Lite and ready for optimization.

Creating and Training an AI Model: Loading the MNIST Dataset

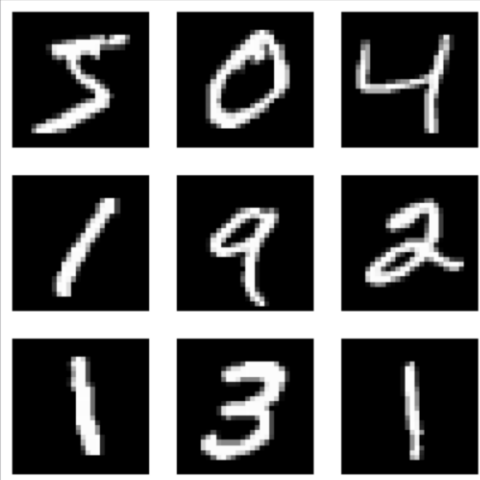
```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
import numpy as np
import matplotlib.pyplot as plt

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize data (scale pixel values between 0 and 1)
x_train, x_test = x_train / 255.0, x_test / 255.0

# Show sample images
plt.figure(figsize=(5,5))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_train[i], cmap="gray")
    plt.axis('off')
plt.show()
```

```
... Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 0s 0us/step
```



Model Conversion

Next, we used the TensorFlow Lite converter to transform the TensorFlow model into a .tflite file. This process reduced the model's size and prepared it for deployment on edge devices.

Converting and Saving the Model: Convert the Model to TensorFlow Lite Format

```
# Load trained model
model = tf.keras.models.load_model("mnist_model.h5")

# Convert to TensorFlow Lite
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

# Save the converted model
with open("mnist_model.tflite", "wb") as f:
    f.write(tflite_model)
print("Model successfully converted and saved as mnist_model.tflite")
```

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be empty until you train or evaluate
Saved artifact at '/tmp/tmpmh7n394n'. The following endpoints are available:
```

```
* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 28, 28), dtype=tf.float32, name='input_layer')
Output Type:
  TensorSpec(shape=(None, 10), dtype=tf.float32, name=None)
Captures:
140069381300560: TensorSpec(shape=(), dtype=tf.resource, name=None)
140069381299408: TensorSpec(shape=(), dtype=tf.resource, name=None)
140069381300176: TensorSpec(shape=(), dtype=tf.resource, name=None)
140069381293072: TensorSpec(shape=(), dtype=tf.resource, name=None)
Model successfully converted and saved as mnist_model.tflite
```

Model Optimization

During conversion, optimization techniques were applied to improve efficiency. These optimizations help reduce memory usage and improve performance while keeping the model functional. This step is important for real-world applications where resources are limited.

Loading and Running Inference with TensorFlow Lite: Load the Converted Model Using TensorFlow Lite Interpreter

```
# Load TensorFlow Lite model
interpreter = tf.lite.Interpreter(model_path="mnist_model.tflite")
interpreter.allocate_tensors()

# Get input and output tensor details
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

print("Input Details:", input_details)
print("Output Details:", output_details)

# Select a random sample from the test set
sample_index = np.random.randint(0, len(x_test))
sample_image = x_test[sample_index]
sample_true_label = y_test[sample_index]

# Prepare the input data for the TFLite model
# The input tensor expects a specific shape and data type
input_shape = input_details[0]['shape']
input_data = np.expand_dims(sample_image, axis=0).astype(np.float32)

# Set the tensor for input
interpreter.set_tensor(input_details[0]['index'], input_data)

# Invoke the interpreter
interpreter.invoke()
```

Inference Testing

After conversion, we loaded the TensorFlow Lite model using the interpreter and ran inference on sample input data. The results showed that the model produced valid outputs, confirming the conversion was successful.

Performing Inference with TensorFlow Lite

```
# Select a test image
test_image = x_test[0].astype(np.float32) # Ensure data type matches model input
test_image = np.expand_dims(test_image, axis=0) # Reshape for model input

# Set the input tensor
interpreter.set_tensor(input_details[0]['index'], test_image)

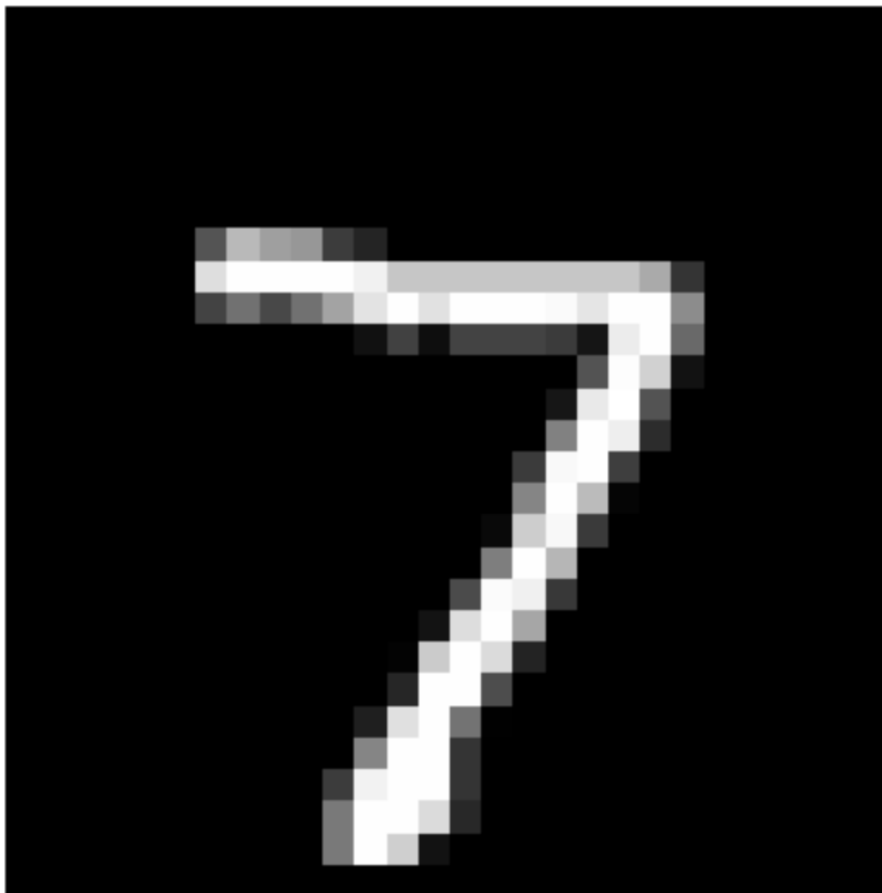
# Run inference
interpreter.invoke()

# Get the prediction
output_data = interpreter.get_tensor(output_details[0]['index'])
predicted_label = np.argmax(output_data)

# Display the image and prediction
plt.imshow(x_test[0], cmap="gray")
plt.title(f"Predicted Label: {predicted_label}, Actual Label: {y_test[0]}")
plt.axis('off')
plt.show()
```

...

Predicted Label: 7, Actual Label: 7



Conclusion

This lab demonstrated how a TensorFlow model can be converted and optimized using TensorFlow Lite. The process showed how machine learning models can be adapted for real-world deployment on resource-constrained devices.